# INFORME DE LABORATORIO

# (formato estudiante)

| INFORMACIÓN BÁSICA | | | | | |
|---|---|---|---|---|---|
| **ASIGNATURA:** | INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E) | | | | |
| **TÍTULO DE LA PRÁCTICA:** | Plano interactivo de un ambiente con Canvas en Jetpack Compose | | | | |
| **NÚMERO DE PRÁCTICA:** | 7 | **AÑO LECTIVO:** | 2026-B | **NRO. SEMESTRE:** | VIII |
| **FECHA DE PRESENTACIÓN** | 17/10/2026 | **HORA DE PRESENTACIÓN** | 23:59 | | |
| **INTEGRANTE (s):**<br>- Zuñiga Villacorta, Peter Sebastian | | | | **NOTA (0-20)** | |
| **DOCENTE(s):** Roxana Evelyn Limache Calatayud | | | | | |

## RESULTADOS Y PRUEBAS

**Github Repo:** https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab07

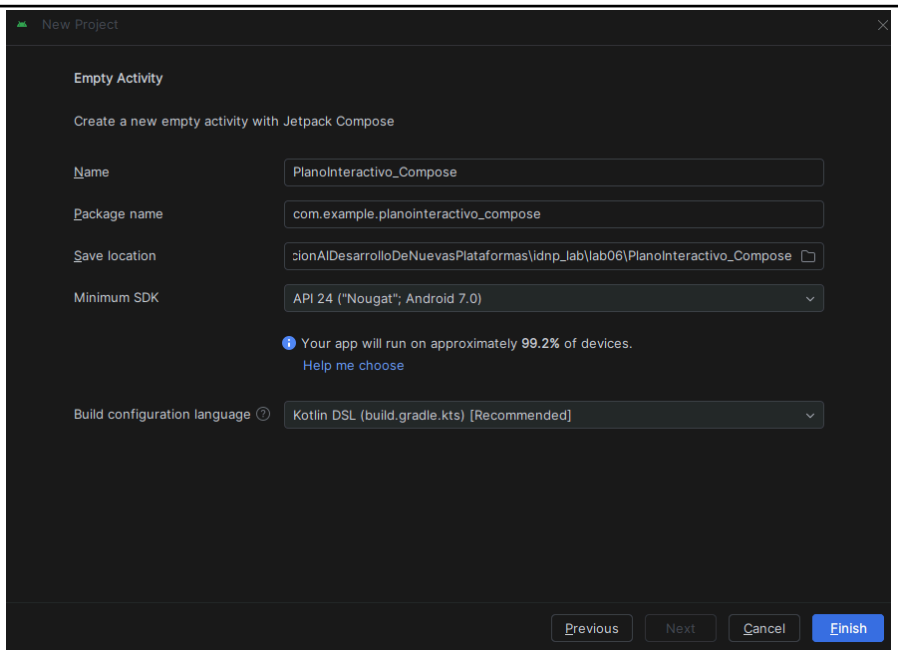
### 1. OBJETIVOS

- Comprender el Composable Canvas y las funciones de dibujo (DrawScope) disponibles en Jetpack Compose.
- Implementar el plano de un ambiente dibujando sus salas mediante primitivas de Canvas y marcadores interactivos.
- Detectar toques sobre regiones específicas del Canvas y mostrar información adicional al seleccionar un marcador.

### 2. EJERCICIOS RESUELTOS

A continuación se detallan, paso a paso, las instrucciones para dibujar el plano de un ambiente con dos salones y marcadores interactivos que muestran información al ser tocados.

**Paso 1:** Crear un nuevo proyecto.




**Paso 2:** Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio:

Kotlin

```
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import com.example.planointeractivo_compose.ui.theme.PlanoInteractivo_ComposeTheme
```

**Paso 3:** Definir, fuera de cualquier Composable, la data class que representará cada marcador del plano:

```kotlin
data class Marcador(
    val offset: Offset,
    val radio: Float,
    val titulo: String,
    val descripcion: String
)
```

**Paso 4:** Crear el Composable PlanoAmbiente, con una lista de al menos 2 marcadores de muestra (remember) y una variable de estado para el marcador seleccionado. Por ahora dejamos el resto del cuerpo vacío; lo iremos completando en los siguientes pasos:

```kotlin
@Composable
fun PlanoAmbiente() {
    val marcadores = remember {
        listOf(
            Marcador(Offset(100f, 150f), 40f, "Sala A", "Sala de cómputo 1"),
            Marcador(Offset(250f, 150f), 40f, "Sala B", "Sala de cómputo 2")
        )
    }
    var marcadorSeleccionado by remember { mutableStateOf<Marcador?>(null) }

   // el Canvas se agrega en el Paso 5
}
```

**Paso 5:** Declarar un Canvas que ocupe el ancho disponible con una altura fija, agregando ya el Modifier.pointerInput con detectTapGestures (todavía vacío). Dentro del Canvas dejamos, por ahora, un comentario donde irá el dibujo:

```kotlin
@Composable
fun PlanoAmbiente() {
    val marcadores = remember {
        listOf(
            Marcador(Offset(100f, 150f), 40f, "Sala A", "Sala de cómputo 1"),
            Marcador(Offset(250f, 150f), 40f, "Sala B", "Sala de cómputo 2")
        )
    }
    var marcadorSeleccionado by remember { mutableStateOf<Marcador?>(null) }

    Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
```

```
            .pointerInput(Unit) {
                detectTapGestures { offset ->
                    // aquí se calcula el marcador tocado, en el Paso 8
                }
            }
    ) {
       // aquí se dibuja el plano, en los Pasos 6 y 7
    }
}
```

**Paso 6:** Dentro del Canvas, dibujar los dos salones con drawRect y la pared divisoria entre ellos con drawLine:
**Paso 7:** Dentro del mismo Canvas, recorrer la lista de marcadores dibujando cada uno con drawCircle. Con esto, el dibujo del plano queda completo:

Kotlin

```
Canvas(
        modifier = Modifier
            .fillMaxWidth()
            .height(300.dp)
            .pointerInput(Unit) {
                detectTapGestures { offset ->
                     // aquí se calcula el marcador tocado, en el Paso 8
                    }
                }
            }
    ) {
        // Salón 1
        drawRect(
            color = Color(0xFFBBDEFB),
            topLeft = Offset(20f, 20f),
            size = Size(150f, 250f)
        )
        // Salón 2
        drawRect(
            color = Color(0xFFC8E6C9),
            topLeft = Offset(180f, 20f),
            size = Size(150f, 250f)
        )
        // Pared divisoria
        drawLine(
            color = Color.DarkGray,
            start = Offset(170f, 20f),
            end = Offset(170f, 270f),
            strokeWidth = 4f
        )
        // Marcadores
        marcadores.forEach { marcador ->
```

```
            drawCircle(
                color = Color.Red,
                radius = marcador.radio,
                center = marcador.offset
            )
        }
    }
```

**Paso 8:** Dentro de detectTapGestures, recorrer la lista de marcadores y calcular la distancia entre el toque y el centro de cada marcador; si es menor o igual al radio, asignarlo a marcadorSeleccionado. Con esto, el Canvas queda completo:

Kotlin

```
.pointerInput(Unit) {
                detectTapGestures { offset ->
                    marcadores.forEach { marcador ->
                        if ((offset - marcador.offset).getDistance() <= marcador.radio) {
                            marcadorSeleccionado = marcador
                        }
                    }
                }
            }
```

**Paso 9:** Fuera del Canvas, mostrar un AlertDialog cuando marcadorSeleccionado no sea null, con el título y la descripción del marcador. Con esto, PlanoAmbiente queda completo:

Kotlin

```
// Diálogo informativo
    marcadorSeleccionado?.let { marcador ->
        AlertDialog(
            onDismissRequest = { marcadorSeleccionado = null },
            title = { Text(marcador.titulo) },
            text = { Text(marcador.descripcion) },
            confirmButton = {
                TextButton(onClick = { marcadorSeleccionado = null }) {
                    Text("Cerrar")
                }
            }
        )
    }
```

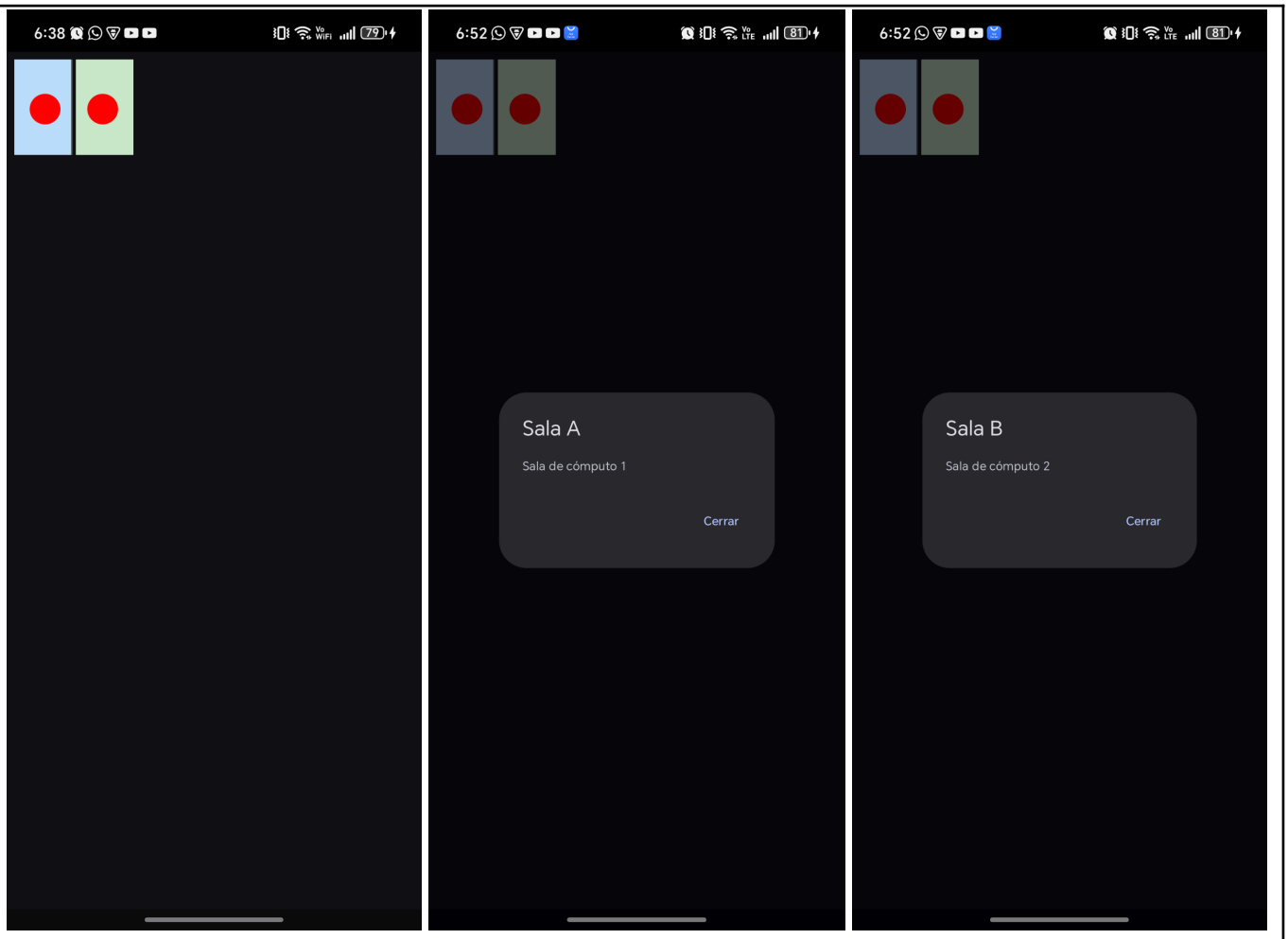
**Paso 10:** Dentro de MainActivity, en el bloque setContent, mostrar PlanoAmbiente:

Kotlin

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            PlanoInteractivo_ComposeTheme {
                Surface(modifier = Modifier.fillMaxSize()) {
                    PlanoAmbiente()
                }
            }
        }
    }
}

@Preview(showBackground = true)
@Composable
fun PlanoAmbientePreview() {
    PlanoInteractivo_ComposeTheme {
        PlanoAmbiente()
    }
}
```

**Paso 11:** Ejecutar la aplicación en un emulador y verificar que, al tocar cada marcador dibujado sobre el plano, se abre el diálogo correspondiente con su información.




## 3. EJERCICIOS PROPUESTOS|

**Amplía el plano resuelto por el docente para representar la visualización interior de al menos una edificación del campus:**

1. **Amplía la lista de marcadores y agrega los rectángulos/líneas necesarios para representar al menos 4 ambientes (salones y/o patios) de una edificación real del campus, cada uno con su etiqueta visible en el plano.**

Se amplió la estructura del plano para representar el Pabellón de Sistemas con 4 ambientes (Lab LIS, Lab LIDS, Aula 101, Patio Central). Las etiquetas visibles fueron renderizadas utilizando drawText mediante rememberTextMeasurer():

Kotlin

```
// Renderizado de ambiente y etiqueta (Ejemplo Lab LIS)
drawRect(color = Color(0xFFBBDEFB), topLeft = Offset(20f, 20f), size = Size(160f, 130f))
drawText(
    textMeasurer = textMeasurer,
    text = "Lab LIS",
    topLeft = Offset(30f, 30f),
```

```
    style = TextStyle(color = Color(0xFF0D47A1), fontSize = 13.sp, fontWeight =
FontWeight.Bold)
)
```

2. **Convierte la lista de marcadores en datos cargados desde un archivo (por ejemplo un .json o .csv en assets/) en lugar de estar escritos directamente en el código, siguiendo un patrón de persistencia similar al usado en el Laboratorio 03.**

Se configuró el archivo app/src/main/assets/marcadores.json y se implementó la lectura con JSONArray emulando la técnica de lectura de archivos del Laboratorio 03:

MainActivity.kt | build.gradle.kts (:app) | marcadores.json

```
[
  {
    "x": 100.0,
    "y": 85.0,
    "radio": 35.0,
    "titulo": "Lab LIS",
    "descripcion": "Laboratorio de Ingeniería de Software equipado con 30 estaciones de trabajo."
  },
  {
    "x": 280.0,
    "y": 85.0,
    "radio": 35.0,
    "titulo": "Lab LIDS",
    "descripcion": "Laboratorio de Investigación y Desarrollo de Sistemas informáticos."
  },
  {
    "x": 100.0,
    "y": 235.0,
    "radio": 35.0,
    "titulo": "Aula 101",
    "descripcion": "Aula teórica principal para clases magistrales de la EPIS."
  },
  {
    "x": 280.0,
    "y": 235.0,
    "radio": 35.0,
    "titulo": "Patio Central",
    "descripcion": "Área de integración y descanso para los estudiantes del pabellón."
  }
]
```

```kotlin
fun cargarMarcadoresDesdeAssets(context: Context): List<Marcador> {
    val lista = mutableListOf<Marcador>()
    try {
        val jsonString = context.assets.open("marcadores.json").bufferedReader().use {
it.readText() }
        val jsonArray = JSONArray(jsonString)
        for (i in 0 until jsonArray.length()) {
            val obj = jsonArray.getJSONObject(i)
            lista.add(Marcador(
                Offset(obj.getDouble("x").toFloat(), obj.getDouble("y").toFloat()),
                obj.getDouble("radio").toFloat(),
                obj.getString("titulo"),
                obj.getString("descripcion")
            ))
        }
    } catch (e: Exception) { e.printStackTrace() }
    return lista
}
```

3. **Traslada el estado del plano y sus marcadores a un ViewModel (PlanoViewModel), siguiendo el patrón de ViewModel visto en el Laboratorio 05, en lugar de mantenerlo directamente en el Composable.**

Siguiendo la arquitectura del Laboratorio 05, la gestión de estados se migró a PlanoViewModel mediante StateFlow:

```kotlin
class PlanoViewModel : ViewModel() {
    private val _marcadores = MutableStateFlow<List<Marcador>>(emptyList())
    val marcadores: StateFlow<List<Marcador>> = _marcadores.asStateFlow()

    private val _marcadorSeleccionado = MutableStateFlow<Marcador?>(null)
    val marcadorSeleccionado: StateFlow<Marcador?> = _marcadorSeleccionado.asStateFlow()

    fun inicializarMarcadores(context: Context) {
        if (_marcadores.value.isEmpty()) {
            _marcadores.value = cargarMarcadoresDesdeAssets(context)
        }
    }

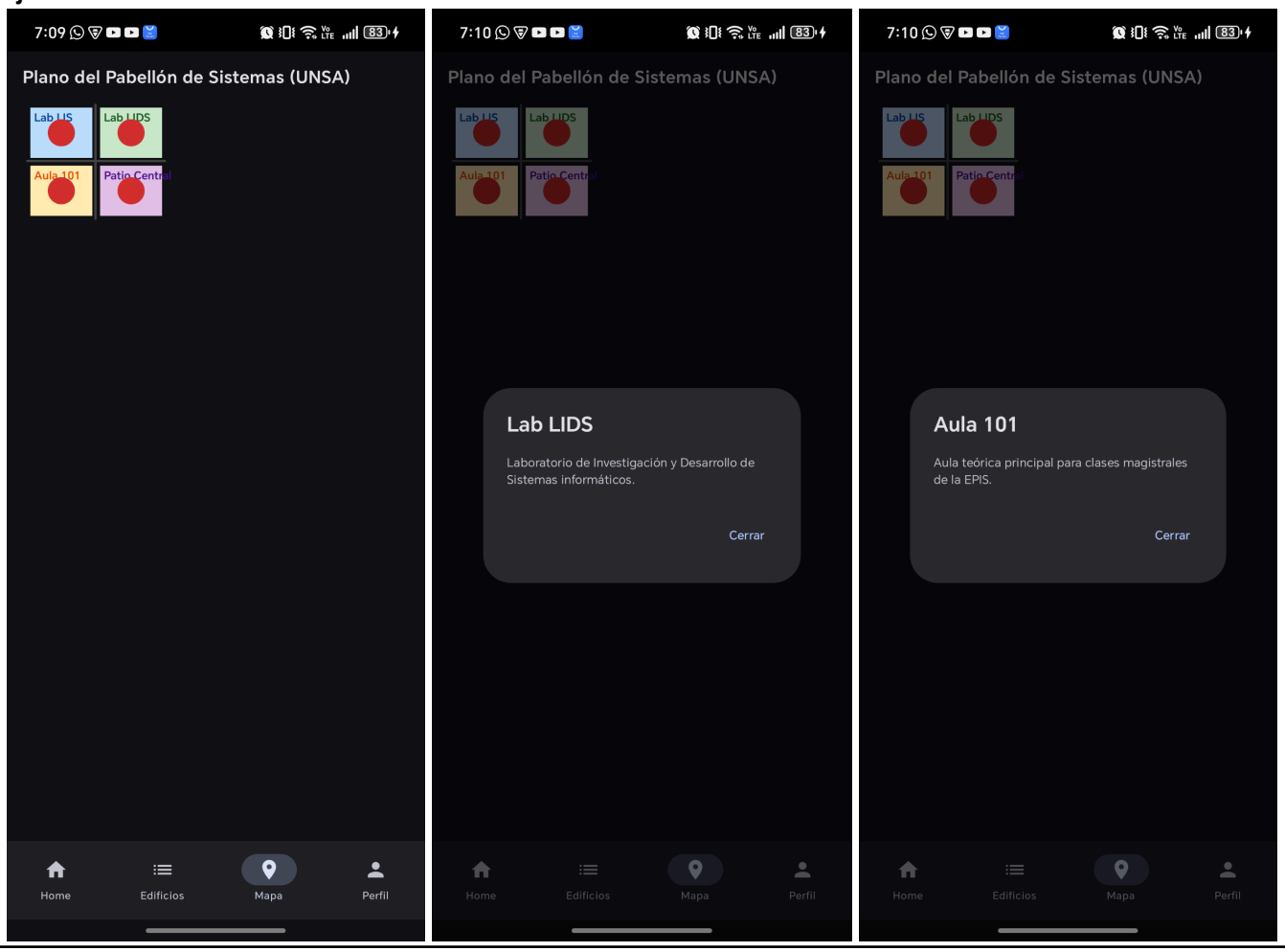
    fun seleccionarMarcador(marcador: Marcador?) {
        _marcadorSeleccionado.value = marcador
    }
}
```

4. **Integra este plano como una nueva pantalla accesible desde MapaScreen (Laboratorio 05) o desde una tarjeta del Laboratorio 06.**

Se vinculó la pantalla del plano en la barra de navegación inferior del contenedor principal (MainScreen) bajo la ruta Screen.Mapa.route:

```
Kotlin
NavHost(
            navController = navController,
            startDestination = Screen.Mapa.route,
            modifier = Modifier.padding(padding)
        ) {
            composable(Screen.Home.route) { Text("Pantalla de Inicio", modifier =
Modifier.padding(16.dp)) }
            composable(Screen.Edificios.route) { Text("Lista de Edificios", modifier =
Modifier.padding(16.dp)) }
            composable(Screen.Mapa.route) { PlanoAmbiente() }
            composable(Screen.Perfil.route) { Text("Perfil de Usuario", modifier =
Modifier.padding(16.dp)) }
        }
```

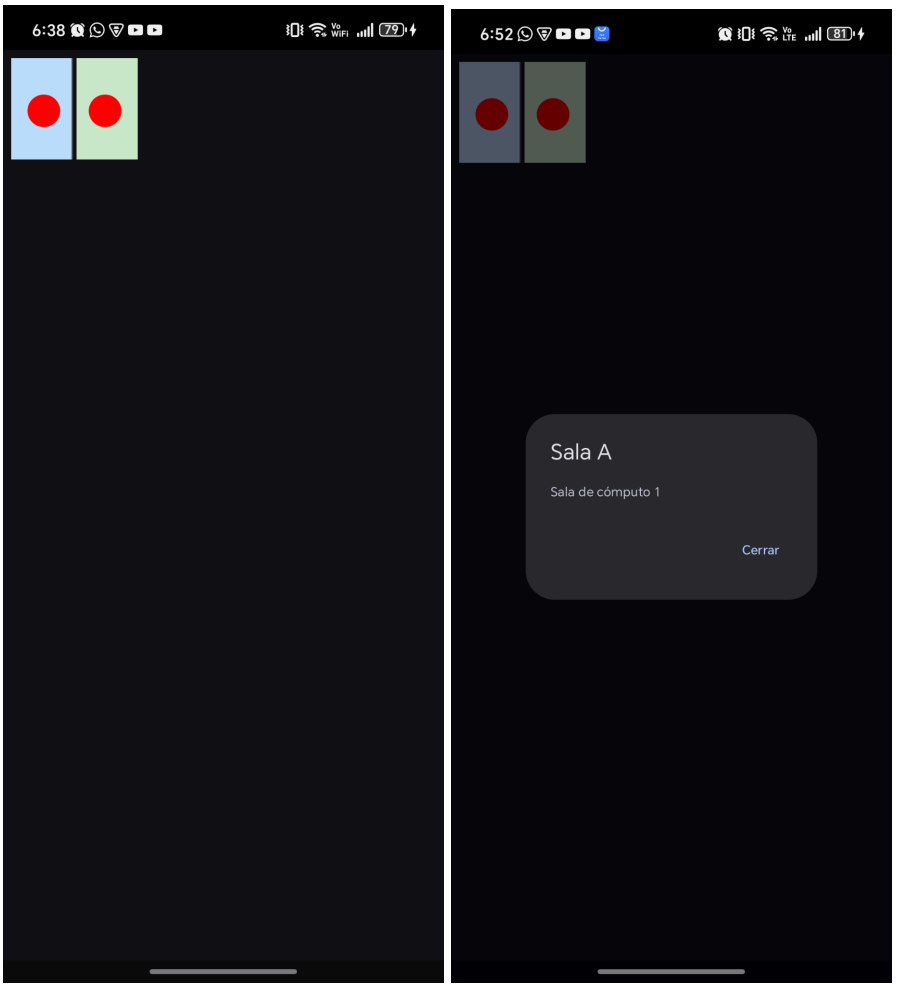
**Ejecución:**



## CUESTIONARIO:

1. **¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (plano de un ambiente dibujado con Canvas, donde tocar un marcador abre un diálogo con su información)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.**

Sí, se completó al 100% la implementación del ejercicio resuelto siguiendo rigurosamente los 11 pasos planteados por el docente. El Canvas dibuja correctamente los dos salones (colores azul claro y verde claro), la línea divisoria gris de separación y los círculos rojos que representan los marcadores de las salas A y B. Al presionar sobre cualquiera de estos círculos, el detector de gestos captura las coordenadas, calcula la colisión y abre un modal AlertDialog mostrando el título de la sala y su correspondiente descripción.




2. **En tu propia implementación, ¿cómo calculaste si un toque cayó dentro de un marcador circular? ¿Qué otra estrategia se te ocurre para detectar toques sobre formas no circulares, como un rectángulo o un polígono?**

**Cálculo para marcador circular:**

En la implementación se utilizó la fórmula de distancia euclidiana entre el punto donde el usuario tocó la pantalla ($P_{toque} = (x_t, y_t)$) y el punto central del marcador ($P_{centro} = (x_c, y_c)$):

$$Distancia = \sqrt{(x_t - x_c)^2 + (y_t - y_c)^2}$$

En Kotlin Compose, esto se ejecuta de manera directa mediante la función de extensión .getDistance() sobre la resta de ambos vectores Offset:

```kotlin
val distancia = (offset - marcador.offset).getDistance()

val estaDentro = distancia <= marcador.radio
```

Si la distancia resulta ser menor o igual al radio del marcador, se confirma la colisión.

**Estrategias para formas no circulares:**

A. **Rectángulos (Bounding Box / AABB):**
Para un rectángulo con esquina superior izquierda $(x_{min}, y_{min})$ y dimensiones $(w, h)$, se verifica si las coordenadas del toque cumplen las desigualdades:

$$x_{min} \leq x_{toque} \leq x_{min} + wyy_{min} \leq y_{toque} \leq y_{min} + h$$

En Jetpack Compose, se puede verificar utilizando la clase Rect con rect.contains(offset).

B. **Polígonos Irregulares (Algoritmo Ray-Casting / Point in Polygon):**
Consiste en trazar un rayo imaginario desde el punto del toque $(x_t, y_t)$ extendiéndose horizontalmente hacia el infinito y contar cuántas veces cruza las aristas del polígono. Si el número de intersecciones es **impar**, el punto está **dentro** del polígono; si es **par**, está **fuera**.

C. **Uso de la clase Path y Region (Android Graphics):**
Se puede definir la figura mediante un objeto androidx.compose.ui.graphics.Path y luego convertirlo a un android.graphics.Region utilizando region.setPath(). Posteriormente, se invoca region.contains(x, y) para realizar la verificación gráfica.

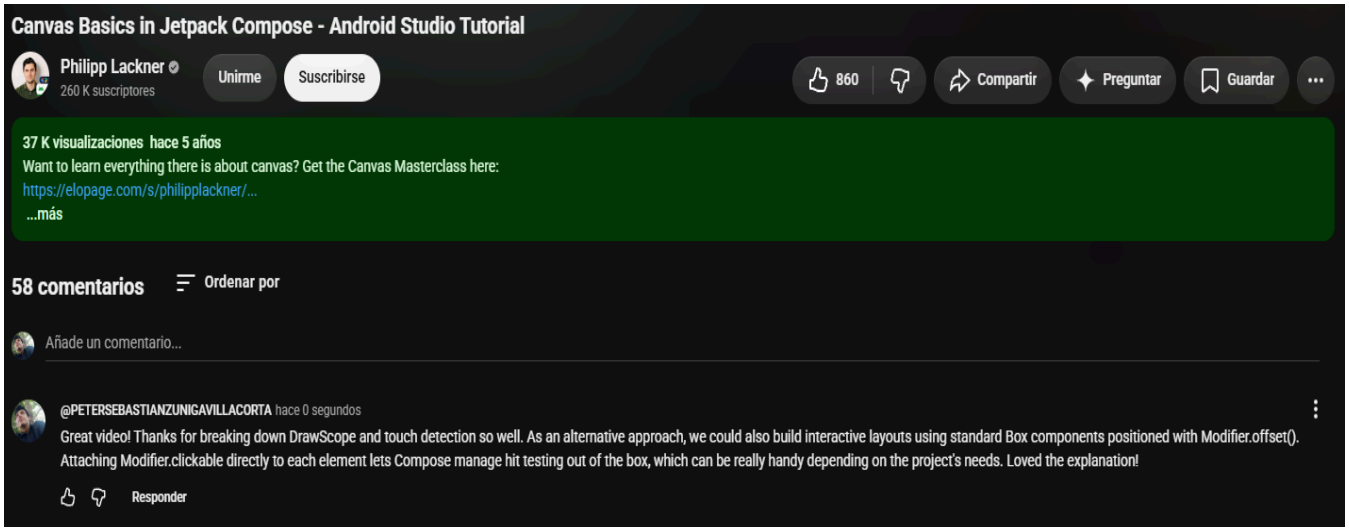
D. **Mapa de Colores / Hit-Testing en Buffer Oculto (Color-Picking):**
Consiste en dibujar en una imagen/bitmap en memoria (no visible) una réplica del plano donde cada ambiente tiene asignado un color RGB único e inconfundible. Al hacer toque en $(x, y)$, se lee el color del píxel del bitmap en esa coordenada y se consulta a qué ambiente pertenece dicho color.

3. **En forma individual, comenta en el video de referencia (https://www.youtube.com/watch?v=cCIPYW_N12o) qué otra forma se te ocurre para lograr un dibujo interactivo similar sin depender directamente de Canvas y DrawScope. (Colocar la captura del comentario; uno por integrante del grupo).**

**Alternativa propuesta:**

Una alternativa nativa en Jetpack Compose consiste en construir la planta arquitectónica utilizando un contenedor Box raíz y posicionar los elementos (habitaciones, paredes y marcadores) mediante composables tradicionales (Box, Surface, Card, Image) utilizando modificadores de tamaño (Modifier.size(), Modifier.fillMaxWidth()) y de posición relativa Modifier.offset(x.dp, y.dp).

De esta forma, cada habitación o marcador circular se representa como un elemento UI individual dotado directamente con su propio modificador Modifier.clickable { ... } o Modifier.combinedClickable. Esta aproximación delega la gestión de toques al sistema de eventos táctiles de Android Compose, eliminando por completo la necesidad de realizar cálculos matemáticos manuales de distancias o colisiones vectoriales.




## CONCLUSIONES

1. Se comprendió el funcionamiento del composable Canvas y su contexto de renderizado DrawScope en Jetpack Compose, identificando cómo las funciones de dibujo de bajo nivel permiten construir interfaces gráficas personalizadas e independientes de la jerarquía convencional de vistas de Android.

2. Se implementó de manera precisa la representación bidimensional de los ambientes del campus dibujando sus salas y límites estructurales mediante primitivas vectoriales (drawRect, drawLine, drawCircle y drawText), logrando estructurar un plano arquitectónico claro y escalable con sus respectivos marcadores.

3. Se logró capturar y procesar eventos táctiles sobre regiones específicas del lienzo mediante el modificador pointerInput y detectTapGestures, aplicando cálculos de distancia vectorial para validar la selección de cada marcador y desplegar dinámicamente cuadros de diálogo informativos (AlertDialog).

## REFERENCIAS Y BIBLIOGRAFÍA

[1] Android Developers, "Graphics in Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/graphics/draw/overview

[2] Android Developers, "Pointer input in Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/touch-input/pointer-input

[3] "Canvas Basics in Jetpack Compose - Android Studio Tutorial", video. [En línea]. Disponible: Canvas Basics in Jetpack Compose - Android Studio Tutorial